Nama : Davyn Xyello Cristwant (Alex)
Kelas : TI-1G
No. Presensi : 2

PERCOBAAN 1

1. Program akan mengalami eror. Karena array bil dideklarasikan dengan tipe data int (bilangan bulat), sehingga tidak dapat menampung nilai pecahan/desimal (double), meskipun untuk angka besar seperti 2000000 muat dalam int.

2. Inisialisasi sekaligus saat deklarasi: Dapat langsung dituliskan menggunakan kurung kurawal tanpa new int[], contohnya: int[] bil = {5, 13, -7, 17};

3. Keluaran statement System.out.println("bil[" + i + "] = " + bil[i] + ", double = " + (bil[i] * 2)): Program akan mencetak nilai asli elemen array beserta hasil perkalian nilai tersebut dengan 2. Contoh: bil[0] = 5, double = 10.

4. Jika kondisi perulangan diubah menjadi i <= 4: Program akan menghasilkan error. Alasannya, array berkapasitas 4 memiliki indeks dari 0 hingga 3. Ketika i = 4, program mencoba mengakses bil[4] yang berada di luar batas alokasi memori array.

5. For loop: Digunakan ketika kita memerlukan akses ke indeks array (misalnya untuk manipulasi atau menampilkan posisi indeks).
   Foreach loop: Digunakan untuk membaca seluruh elemen array secara berurutan tanpa memerlukan nomor indeks.
   Arrays.toString(): Digunakan untuk mencetak seluruh isi array dalam satu baris dengan format standar.

PERCOBAAN 2

1. Berfungsi sebagai batasan perulangan agar berjalan sebanyak ukuran panjang array (10 elemen), mencegah program mengakses indeks di luar batas.

2. Untuk menampilkan yang lulus saja, tambahkan seleksi kondisi if (nilaiAkhir[i] > 70) di dalam perulangan *output*, sehingga baris dengan nilai ≤ 70 dilewati dan tidak dicetak ke layar.

3. Bagian *input* **tidak bisa** diubah menggunakan *foreach* karena *foreach* bersifat *read-only* (hanya membaca) dan tidak dapat mengakses indeks array untuk menyimpan data inputan (nilaiAkhir[i] = ...). Bagian *output* bisa diubah ke *foreach* asalkan kita tidak memerlukan informasi nomor indeks (i + 1).

4. Dapat dilakukan dengan menyediakan dua variabel penghitung (misal jumlahLulus dan jumlahTidakLulus) yang nilainya bertambah +1 di dalam percabangan if-else ketika status kelulusan dicek.

PERCOBAAN 3

1. Dapat diselesaikan dengan membuat variabel counter yang bertambah jika nilaiMhs[i] > 70 di dalam perulangan pengecekan.

2. Dilakukan dengan meminta input integer terlebih dahulu sebelum inisialisasi array, sehingga ukuran array dinamis (int[] nilaiMhs = new int[jumlahMhs];).

3. Karena variabel total dan nilaiMhs.length bertipe data int. Jika pembagian dilakukan antarsesama integer (int / int), Java akan melakukan pembagian bulat (integer division) yang menghilangkan bagian desimalnya. Penambahan (double) melakukan casting agar hasil operasi pembagian menghasilkan bilangan pecahan yang akurat

PERCOBAAN 4

1. Berfungsi untuk menghentikan perulangan secara paksa segera setelah nilai yang dicari (key) ditemukan, sehingga program tidak perlu membuang waktu memproses elemen-elemen selanjutnya.

2. Sudah diterapkan pada kode program di atas dengan menggunakan Scanner untuk menentukan jumlah elemen dan isi array sesuai input pengguna.

3. Sudah diimplementasikan di kode program.

4. Algoritma ini bekerja dengan membandingkan dua elemen yang bersebelahan secara berpasangan. Jika posisinya tidak sesuai urutan (misalnya elemen kiri lebih besar daripada elemen kanan untuk *ascending*), maka posisinya ditukar (*swap*). Proses ini diulang terus menerus secara bertahap hingga seluruh elemen terurut sempurna.

5. Dilakukan dengan membalik tanda perbandingan pada kondisi if dari nilai[j] > nilai[j + 1] menjadi nilai[j] < nilai[j + 1].